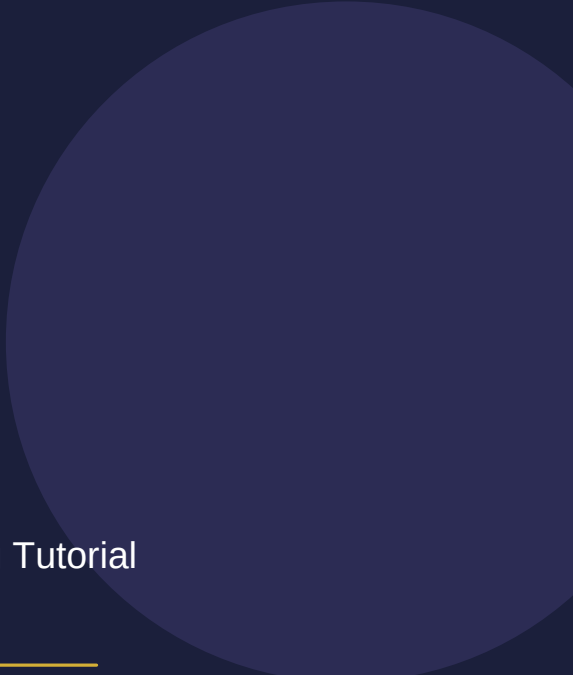




PySpark

Step-by-Step Big Data Engineering Tutorial

From SparkSession to Parquet & CSV Output

Dataset: gold.dim_customers.csv | Medallion Architecture – Gold Layer

Covers: DataFrame API • Spark SQL • Window Functions • Joins • File I/O

Table of Contents

01

02

03

04

05

06

07

08

09

10

11

12

This tutorial is a hands-on, beginner-to-intermediate path into PySpark for big-data engineering. Every step pairs runnable code (left) with a plain-language explanation (right) of what the code does and why it matters. Work through the sections in order — each builds on the DataFrame created in the previous one.

01 Setting Up: Creating a SparkSession

Every PySpark program starts by creating a **SparkSession** — the single entry point for working with DataFrames, Spark SQL, and reading/writing files. Think of it as opening the connection to the Spark engine before you can do anything else.

CODE	EXPLANATION
<pre># Install (once, in your environment) # pip install pyspark from pyspark.sql import SparkSession spark = (SparkSession.builder .appName("GoldDimCustomersTutorial") .master("local[*]") .config("spark.sql.shuffle.partitions", "4") .getOrCreate()) print(spark.version)</pre>	SparkSession.builder starts a configuration chain. appName() labels the job in the Spark UI/logs. master("local[*"]) runs Spark locally using all CPU cores — in a real cluster this would instead point to YARN, Kubernetes, or a standalone master. getOrCreate() reuses an existing session if one exists, otherwise creates a new one. Setting shuffle.partitions low (4) avoids excessive tiny partitions on small local datasets.

■ **Note:** In notebooks (Databricks, Jupyter with a Spark kernel) a **SparkSession** named **spark** is often already created for you — check before creating a second one.

02 Loading the Dataset into a DataFrame

PySpark DataFrames are the core data structure — a distributed, table-like collection of rows with named, typed columns. Here we load **gold.dim_customers.csv**, a customer dimension table typical of the **gold layer** in a medallion architecture (raw → bronze → silver → gold).

CODE	EXPLANATION
<pre># Option A: let Spark infer the schema (quick, but slower & less safe) df = (spark.read .option("header", True) .option("inferSchema", True) .csv("gold.dim_customers.csv"))</pre>	option("header", True) tells Spark the first row holds column names. inferSchema makes Spark scan the data once to guess types (int, string, date...). This is convenient while exploring, but for production pipelines it's slower on large files and can guess wrong (e.g. reading a date as a string).
<pre># Option B: define an explicit schema (recommended for pipelines) from pyspark.sql.types import (StructType, StructField, IntegerType, StringType, DateType) schema = StructType([StructField("customer_key", IntegerType(), True), StructField("customer_id", StringType(), True), StructField("first_name", StringType(), True), StructField("last_name", StringType(), True), StructField("marital_status", StringType(), True), StructField("gender", StringType(), True), StructField("birthdate", DateType(), True), StructField("country", StringType(), True), StructField("create_date", DateType(), True),]) df = (spark.read .option("header", True) .schema(schema) .csv("gold.dim_customers.csv"))</pre>	An explicit StructType schema skips the inference scan (faster on big files), guarantees consistent types every run, and fails fast/loudly if the source file's structure changes unexpectedly — a best practice for real data-engineering pipelines.

03 Exploring the DataFrame

Before transforming data, always look at its shape, schema, and a sample of rows. These are the PySpark equivalents of pandas' `.head()`, `.info()` and `.describe()`.

CODE	EXPLANATION
<code>df.printSchema()</code>	Prints the column names, data types, and nullability tree — the fastest way to sanity-check that the CSV loaded with the types you expect.
<code>df.show(5, truncate=False)</code>	Displays the first 5 rows as a formatted table. <code>truncate=False</code> stops long strings being cut off with '...'. <code>df.show()</code> is equivalent to <code>df.show(5, truncate=True)</code> .
<code>print("Rows:", df.count())</code> <code>print("Columns:", len(df.columns))</code>	<code>count()</code> triggers a full pass over the distributed data to get the exact row count — an <i>action</i> , unlike most DataFrame methods which are lazy <i>transformations</i> .
<code>df.describe(["customer_key"]).show()</code>	Produces summary statistics (count, mean, stddev, min, max) for numeric columns, similar to pandas' <code>.describe()</code> .
<code>df.columns</code> <code>df.dtypes</code>	<code>.columns</code> returns a plain Python list of column names. <code>.dtypes</code> returns (name, type) tuples — handy for quick programmatic checks.

■ **Key PySpark concept:** Most DataFrame operations (*select*, *filter*, *withColumn*...) are **lazy transformations** — Spark just builds an execution plan. Nothing actually runs until an **action** like *show()*, *count()*, or *write()* is called.

04 Selecting & Filtering Data

Narrowing down columns and rows is the most common first transformation step.

CODE	EXPLANATION
<pre>df.select("customer_id", "first_name", "last_name", "country").show(5)</pre>	select() keeps only the listed columns, discarding the rest — reduces the data volume Spark has to shuffle in later steps.
<pre>from pyspark.sql.functions import col df.filter(col("country") == "Egypt").show(5)</pre>	filter() (alias where()) keeps only rows matching the condition. Using col("country") instead of a raw string is the standard, safest way to reference columns in expressions.
<pre>df.filter((col("country") == "Egypt") & (col("marital_status") == "Married")) .show(5)</pre>	Combine multiple conditions with & (AND) or (OR). Each condition must be wrapped in parentheses due to Python operator precedence.
<pre>df.filter(col("country").isNull()).count()</pre>	isNull() / isNotNull() test for missing values — essential for the data-quality checks any gold-layer table should pass.

05 Adding & Transforming Columns

New, derived columns are created with **withColumn()**. This is the PySpark equivalent of pandas' `df['new_col'] = ...`

CODE	EXPLANATION
<pre>from pyspark.sql.functions import concat_ws, current_date, floor, datediff df = df.withColumn("full_name", concat_ws(" ", col("first_name"), col("last_name")))</pre>	concat_ws(sep, ...) joins columns with a separator, skipping nulls automatically (safer than plain string concatenation).
<pre>df = df.withColumn("age", floor(datediff(current_date(), col("birthdate")) / 365.25))</pre>	datediff() returns the number of days between two dates; dividing by 365.25 and floor() converts that into a whole-number age in years.
<pre>from pyspark.sql.functions import when df = df.withColumn("age_group", when(col("age") < 30, "Young Adult") .when(col("age") < 50, "Adult") .otherwise("Senior"))</pre>	when().otherwise() is Spark's vectorized if/elif/else — the equivalent of np.select() or nested CASE WHEN in SQL.
<pre>df = df.withColumnRenamed("customer_key", "cust_key")</pre>	Renames a single column while keeping all data intact.

06 Handling Missing Values & Duplicates

Gold-layer tables should be clean, but it's still good practice to verify and handle nulls and duplicate rows explicitly.

CODE	EXPLANATION
<pre>df.dropDuplicates(["customer_id"]).count()</pre>	dropDuplicates(subset) removes duplicate rows based on the given column(s) — here, duplicate customer IDs (a common issue after merging source extracts).
<pre>df.na.drop(subset=["country"]).count()</pre>	na.drop(subset=...) removes rows where the specified column(s) are null. Without subset , it drops a row if <i>any</i> column is null.
<pre>df_clean = df.na.fill({ "country": "Unknown", "marital_status": "Unknown" })</pre>	na.fill(dict) replaces nulls per-column with a chosen default — safer than dropping rows when you still need them for counts/aggregations.
<pre>from pyspark.sql.functions import count, when as w df.select([count(w(col(c).isNull(), c)).alias(c) for c in df.columns]).show()</pre>	A common PySpark idiom: build a null-count summary for every column in one pass, similar to pandas' df.isnull().sum() .

07 Aggregations & GroupBy

`groupBy()` + `agg()` is how PySpark summarizes data across groups — the same mental model as pandas' `groupby()` or SQL's **GROUP BY**.

CODE	EXPLANATION
<pre>df.groupBy("country").count().orderBy(col("count")).desc()).show()</pre>	Counts customers per country and sorts descending. count() here is a DataFrame shortcut equivalent to agg(count("*")) .
<pre>from pyspark.sql.functions import avg, min as fmin, max as fmax df.groupBy("country", "gender").agg(avg("age").alias("avg_age"), fmin("age").alias("min_age"), fmax("age").alias("max_age")).orderBy("country").show()</pre>	agg() lets you compute several aggregate functions per group at once. Aliasing (.alias()) is important because Spark auto-generates ugly default column names otherwise (e.g. <code>avg(age)</code>).
<pre>df.groupBy("marital_status").agg(count("*").alias("total")).withColumn("pct", (col("total") / df.count() * 100)).show()</pre>	Combining an aggregation with a follow-up withColumn() shows how transformations chain — here, converting raw counts into percentages of the full customer base.

08 Introduction to Spark SQL

Spark SQL lets you query DataFrames using plain SQL — extremely useful if you (or your team) already think in SQL, and the two APIs are fully interchangeable.

CODE	EXPLANATION
<pre>df.createOrReplaceTempView("dim_customers")</pre>	Registers the DataFrame as a temporary, session-scoped SQL table/view named dim_customers . It isn't written to disk — it only lives for this Spark session.
<pre>spark.sql(""" SELECT country, COUNT(*) AS total_customers FROM dim_customers GROUP BY country ORDER BY total_customers DESC """).show()</pre>	spark.sql() runs standard SQL against the registered view and returns the result as a regular DataFrame — you can keep chaining PySpark methods on it afterward.
<pre>result_df = spark.sql(""" SELECT country, gender, ROUND(AVG(age), 1) AS avg_age FROM dim_customers WHERE country IS NOT NULL GROUP BY country, gender """) result_df.show()</pre>	SQL and DataFrame results are the same underlying object — assign the query output to a variable just like any transformation, then keep using select/filter/write on it.

■ **Rule of thumb:** Use whichever of SQL or the DataFrame API reads more clearly for a given step — Spark's optimizer (Catalyst) compiles both down to the same execution plan, so there's no performance penalty either way.

09 Window Functions

Window functions calculate a value across a set of related rows (a 'window') without collapsing them into a single row, the way `groupBy/agg` does. They're essential for rankings, running totals, and row-over-row comparisons.

CODE	EXPLANATION
<pre>from pyspark.sql.window import Window from pyspark.sql.functions import rank, row_number w = Window.partitionBy("country").orderBy(col("age").desc())</pre>	A Window spec defines: partitionBy (reset the calculation per group, here per country) and orderBy (the order rows are ranked/compared within each partition).
<pre>df.withColumn("age_rank", rank().over(w)) \ .select("country", "first_name", "age", "age_rank") \ .show(10)</pre>	rank().over(w) assigns a rank to each row within its country partition, ordered by age descending — ties share the same rank, with a gap left in the sequence (unlike dense_rank(), which leaves no gap).
<pre>df.withColumn("row_num", row_number().over(w)) \ .filter(col("row_num") == 1) \ .select("country", "first_name", "age") \ .show()</pre>	A very common pattern: use row_number() plus a filter for row_num == 1 to pick the 'top' row per group — here, the oldest customer in each country.
<pre>from pyspark.sql.functions import sum as fsum w_cum = Window.partitionBy("country") \ .orderBy("create_date") \ .rowsBetween(Window.unboundedPreceding, Window.currentRow) df.withColumn("running_count", fsum(col("customer_key")*0+1).over(w_cum)) \ .select("country", "create_date", "running_count").show(10)</pre>	rowsBetween(unboundedPreceding, currentRow) defines a running/cumulative window — here counting how many customers had signed up in each country up to that row's date, a classic 'running total' pattern.

10 Joining DataFrames

Real pipelines almost always combine multiple tables. Here we join the customer dimension against a small lookup DataFrame of country regions.

CODE	EXPLANATION
<pre>region_data = [("Egypt", "MENA"), ("Saudi Arabia", "MENA"), ("United Arab Emirates", "MENA"), ("United States", "Americas"), ("Canada", "Americas"), ("Germany", "Europe"), ("France", "Europe"), ("Australia", "Oceania"),] region_df = spark.createDataFrame(region_data, ["country", "region"])</pre>	spark.createDataFrame() builds a small DataFrame directly from a Python list — handy for lookup/reference tables that don't need to come from a file.
<pre>joined_df = df.join(region_df, on="country", how="left") joined_df.select("customer_id", "country", "region").show(5)</pre>	join(other, on, how) mirrors SQL joins. how="left" keeps every row from df even if no matching region is found (region will be null in that case).
<pre>joined_df.groupBy("region").count().orderBy(col(" count").desc()).show()</pre>	Once joined, the new region column can be used in any further groupBy, filter, or window operation, just like an original column.

■ **Common join types:** *inner* (default, matching rows only), *left/right* (keep all rows from one side), *full* (keep all rows from both sides), *left_semi* (filter df to rows that have a match, keeping only df's columns), *left_anti* (rows in df with no match).

11 Saving Results: Parquet & CSV

The final step of most pipelines is persisting results. Parquet is the standard columnar format for big-data engineering; CSV is often kept for human-readable exports.

CODE	EXPLANATION
<pre>(joined_df.write .mode("overwrite") .partitionBy("region") .parquet("output/dim_customers_parquet"))</pre>	write.parquet() saves the DataFrame in Parquet — a compressed, columnar, schema-aware format that's fast to read back and is what production gold-layer tables typically use. partitionBy("region") writes separate subfolders per region, which speeds up downstream queries that filter by region. mode("overwrite") replaces any existing output at that path.
<pre>(joined_df.write .mode("overwrite") .option("header", True) .csv("output/dim_customers_csv"))</pre>	write.csv() exports to CSV for humans/Excel/other tools. Spark writes one file per partition by default (e.g. <i>part-00000...csv</i>) — use .coalesce(1) beforehand if you need a single output file.
<pre>(joined_df.coalesce(1).write .mode("overwrite") .option("header", True) .csv("output/dim_customers_single_csv"))</pre>	coalesce(1) reduces the DataFrame to a single partition before writing, producing one clean CSV file. Use with care on very large datasets — it forces all data through one task and can be slow/memory-heavy.
<pre>parquet_df = spark.read.parquet("output/dim_customers_parquet") parquet_df.printSchema()</pre>	Reading Parquet back requires no schema definition — the format stores the schema alongside the data, one of the main reasons it's preferred over CSV in pipelines.

12 Stopping the SparkSession & Best Practices

Always release cluster resources when a job finishes, and keep a few habits in mind as you move from local practice scripts to real pipelines.

CODE	EXPLANATION
<pre>spark.stop()</pre>	Ends the SparkSession and releases the resources (executors, memory) it was using. In notebooks this is often optional per-cell, but always required at the end of standalone scripts/jobs.

Best Practices Checklist

1	Cache wisely: use <code>df.cache()</code> only when a DataFrame is reused multiple times — caching everything wastes memory.
2	Avoid collect() on big data: it pulls all rows to the driver's memory; use it only on small, already-aggregated results.
3	Prefer built-in functions (<code>pyspark.sql.functions</code>) over Python UDFs — UDFs break Spark's internal optimizations and run much slower.
4	Watch partition count: too many small partitions (small files problem) or too few large ones both hurt performance — repartition/coalesce deliberately.
5	Read the Spark UI: (usually <code>http://localhost:4040</code> locally) to see stages, tasks, and shuffle sizes when debugging slow jobs.
6	Explicit schemas in production: avoid <code>inferSchema</code> on pipelines that run repeatedly on changing data.

■ You've now covered the full PySpark path: *SparkSession* → *DataFrame loading* → *exploration* → *transformations* → *SQL* → *window functions* → *joins* → *saving to Parquet/CSV*. Next step: *rebuild this same flow against a larger, multi-file dataset on an actual cluster (Databricks / EMR / a local multi-node setup) to see partitioning and shuffles at real scale.*